

Gin Rummy Codebase Reading Order

Recommended order for understanding this codebase. Each layer builds on the previous.

Layer 1: Data Models (Foundation)

1. **card.py** - Start here. Defines **Suit**, **Rank**, and **Card**. Immutable dataclass with deadwood values and comparison operators.
2. **deck.py** - Simple collection of Cards with shuffle/draw operations.
3. **hand.py** - A player's hand of Cards. Has caching for meld analysis.
4. **melds.py** - Core gin rummy logic: sets, runs, and optimal deadwood calculation. The `find_optimal_melds()` function uses recursive backtracking to find the best non-overlapping meld combination.

Layer 2: Configuration

5. **config.py** - Dataclass-based config with TOML loading. Defines game rules, AI behavior, display settings, etc. Most modules import `get_config()` from here.

Layer 3: Game State

6. **player.py** - Thin wrapper: name + hand + score.
7. **game.py** - The state machine. Handles game phases (DEALING, DRAWING, DISCARDING, etc.), enforces rules, tracks discard history. The **Game** class is where gin rummy rules are implemented.

Layer 4: AI System

8. **context.py** - AI support infrastructure:
 - **KnownCards / CardLocation** - Card location tracking
 - **OutsCalculator** - Finds cards that would help a hand
 - **OpponentModel** - Tracks opponent patterns
 - **DynamicThresholdCalculator** - Adjusts draw threshold based on game state
9. **ai.py** - The AI players:
 - **BasicAI** - Simple heuristic (minimize deadwood)
 - **ContextAwareAI** - Extends BasicAI with dynamic thresholds, outs analysis, and opponent modeling

Layer 5: Execution

10. **game_runner.py** - Shared turn execution logic. Both CLI and Simulator use `execute_ai_turn()` from here to avoid code duplication.

Layer 6: Entry Points / Applications

11. **cli.py** - Terminal interface for human vs AI play. Uses ANSI colors, handles input, displays game state.
12. **simulator.py** - AI vs AI with metrics collection. Good for testing AI changes.
13. **database.py** - SQLite schema and **GameTracker** for persisting game history.
14. **analyze_hand.py** - CLI utility to analyze a hand's outs. Usage: `uv run python -m gin_rummy.analyze_hand "3S 8S 2H..."`

Optional: Network

15. **network/** - Client/server for networked multiplayer (protocol.py defines message types, client.py and server.py implement the connection logic).

Dependency Graph (simplified)

